

Ejercicio 1). Policías

Toda esquina debe estar a lo sumo a 5 cuartos de una estación de policía. Dada la lista de esquinas $\{v_1, \dots, v_n\}$ de la ciudad, la lista $\{p_1, \dots, p_k\}$ de esquinas donde hay policías, y la lista E de calles, debemos indicar si la normativa se cumple, y en el caso contrario cuáles esquinas son las que quedan desprotegidas.

Solución.

Agregamos un nodo fantasma z al grafo y lo conectamos con todos los policías.

Corro BFS desde z y veo qué nodos están a distancia mayor a 6.

$O(n+m)$

¿Por qué vale el siguiente lema: un nodo v está a distancia menor o igual a 5 de un policía $\Leftrightarrow v$ está a distancia menor o igual a 6 de z .

Osea: sea z el nodo fantasma y S el cmo de orígenes (policías en este caso).

$$\forall v \neq z, d(v, z) = \min_{s \in S} \{d(s, v)\} + 1$$

Sabemos que $\forall v, d(v, z) = \min_{s \in S} \{d(s, z) + d(s, v)\}$ ya

que z sólo está conectado a los nodos s .

Además, $d(s, z) = 1$ ya que z está conectado a todos los orígenes con una arista. $\Rightarrow \forall v, d(v, z) = \min_{s \in S} \{d(v, s) + 1\}$

Saco el 1 $\Rightarrow \forall v, d(v, z) = \min_{s \in S} \{d(v, s)\} + 1$

Ejercicio 2). Los mares

quiero cruzar el mar tortugoso empezando desde la isla Anastasia, parando en algunas islas para comerciar, y llegando a la isla Betuna. Pero hay peces que se esconden en el barco y pudren la madera. Sabe qué camino debe usar para pasar de una isla a otra y sabe cuántos peces van a esconderse en el barco en cada ruta.

En algunas de esas rutas, hay tortugas que se comen un porcentaje de los peces que tenga el barco.

Quiero pasar por máximo una sola de las rutas con tortugas.

¿Cuál es la mínima cant. de peces con la que puedo alcanzar el otro lado del mar tortugoso?

Solución.

Diagrama D. las islas son los nodos y las rutas son las aristas, con costo $p(u,v)$ para los que suman peces y con porcentaje $t(u,v)$ para las que tienen tortugas.
↳ fracción.

idea: Sacamos del diagrama a las aristas con tortugas. Lo llamamos D' .

en D' las aristas no tienen costos negativos $\Rightarrow$ Dijkstra.
queremos llegar de A a B.

como Dijkstra desde A. $O(\min\{m \log(n), n^2\})$

responderemos D' y corremos Dijkstra desde B.

$O(nm)$

↳ $O(\min\{m \log(n), n^2\})$

Por cada arista tornosa (u, v) podemos calcular cuál sería el costo mínimo de un camino que la contiene.

$$w(A, u) \cdot (1 - t(u, v)) + w(v, B)$$

↓
peso del camino. ↘ fracción que queda después de pasar por la arista.

$w(A, u)$ y $w(v, B)$ lo tenemos precalculado con Dijkstra.

devolvemos el camino con menor peso.

$O(m)$.

Ejercicio 3). MONSTRUOS.

quiero que el jugador pase por distintos mundos pasando por portales unidireccionales, matando los monstruos de los mundos por los que pasa.

Cada portal i por el que logra pasar le da $p_i \in \mathbb{R}$ puntos.

El objetivo del juego es llegar desde el mundo m_1 hasta el mundo m_n con la mayor cant. de puntos posibles.

Quiero saber la Máx. cant. de puntos que puede generar un jugador dada la cant. de mundos, los portales, y cuántos puntos da cada portal.

Además, quiero saber cuál es la min. cant. de portales que podría estar tomando el jugador que hiciese la máx. cant. de puntos posibles.

Solución.

digrafo G . los nodos son los mundos, los ejes son los portales y los pesos de las aristas son el puntaje del portal.

Una partida sería un recorrido R que empiece en m_1 y termine en m_n .

Sup. que no hay ejes salientes de m_n .

Multiplicamos los pesos de las aristas por -1 . (ponerle que sea el recorrido máximo, no el mínimo, y si multiplico por -1 lo convierto en un problema de recorrido mínimo).

De haber ciclos positivos, no hallaría puntaje máximo.

Al multiplicar por -1 , pasan a ser ciclos negativos.

Solo nos importan los ciclos negativos que están en medio de un recorrido de m_1 a m_n .

Para esto, traspongo G , como DFS desde m_n y saco los nodos que no alcanzan a m_n .

Ahora corro Bellman-ford desde m_1 .

devolvemos infinito si encuentra el ciclo. Si no lo encuentra, devolvemos el opuesto del valor del recorrido mínimo entre m_1 y m_n .
↳ pues multiplicamos por -1 .

Queremos ver cuál es la cant. min. de aristas por las que puede obtener ese puntaje máximo.

Queremos pasar por la min. cant. de portales posibles.

Si el puntaje es infinito, la min. cant. de aristas que se necesitan para obtenerlo es infinito.

Veamos cuando no es infinito.

Construimos un DAG de recorridos mínimos de m_1 a m_n .

Este DAG contiene todas las aristas pertenecientes a un recorrido mínimo entre m_1 y m_n .

como Bellman-ford desde m_1 y guardo el resultado en un vector "dist.desdem₁".

traspongo G .

como Bellman-ford desde m_n y guardo el resultado en un vector "dist.Hastam_n".

Por cada arista $(u, v) \in G$, si cumple:
 $\text{dist. desde } m_1(u) + \text{peso}(u, v) + \text{dist. hasta } m_n(v) = \text{dist. desde } m_1(m_n)$
 $\Rightarrow$ agrego (u, v) al DAG de recorridos mínimos.

como DFS en el DAG desde m_1 . La altura de m_n va a ser el largo del recorrido min. más corre

Ejercicio 4). **Sasha y dais**

tenemos n ciudades y m rutas. Cada ruta e_i tiene un cierto peso asociado p_i y se tarda m_i minutos en recorrerla.

Queremos encontrar el camino de A a B que minimice el costo de los peajes a pagar y que se pueda recorrer en menos de t minutos. t es "chico" ($t \in O(n)$).

Solución.

Podemos llevar la cuenta del tiempo en los nodos. Creamos un digrafo G_t donde sus vértices son los pares (c, t') con $c \in V$ y $0 \leq t' \leq t$.

t' sería el tiempo acumulado hasta llegar a c .

Definido así, éste nunca va a ser $> t$. ✓

Conectamos (c_i, t_i) con (c_j, t_j) con un eje de peso p_j
 $\Leftrightarrow t_j = t_i + m_i \leq t$

Agrego un nodo fantasma. A los nodos de la forma (B, t') $\forall t'$ agregamos un eje de costo 0 que lo conecte al nodo fantasma.

Así, queremos encontrar un camino mínimo desde $(A, 0)$ en G_t .

G_t tiene $O(nt)$ nodos y $O(mt)$ ejes

Como Dijkstra es $O(\min\{ (nt)^2, mt \log(nt) \})$

Complejidad : $O(mt \log(nt))$

Otra forma: G_t es un DAG. Sabemos que no tiene ciclos porque t' va aumentando.
Me armo la función de PD:

$$d_A(v) = \begin{cases} 0 & \text{si } A=B \\ \min\{w(B,v) + d_A(B) : B \rightarrow v \in E\} & \text{si no } \textcircled{*} \end{cases}$$

$\textcircled{*}$ caso recursivo: para todos los vecinos, me fijo cuál es el camino mínimo

Así, la complejidad me queda lineal en base al tamaño del grafo: $O(m_t + n_t)$

$\Rightarrow$ me quedaria:

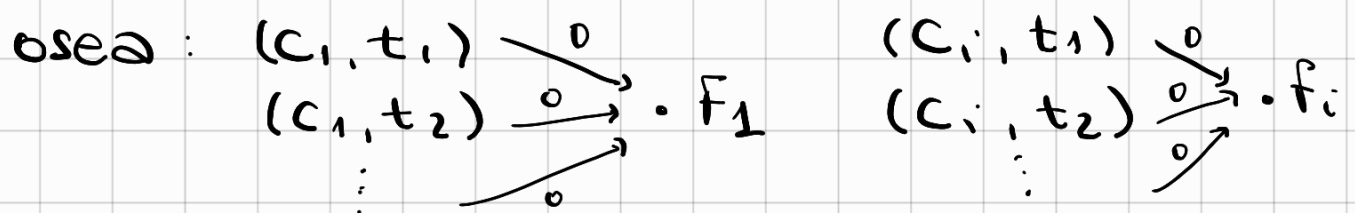
construyo G_t en $O(m_t + n_t)$

calculo el camino mínimo de $(kruskal, 0)$ al nodo farasua en $O(m_t + n_t)$

$\Rightarrow O(m_t + n_t)$

item b). ¿Cómo cambiaría nuestra solución si nos dicen que queremos encontrar la ruta que cumpla las mismas especificaciones pero para todas las ciudades?

Agrego un nodo fantasma conectado con costo 0 a cada ciudad.



Así, como Dijkstra desde $(A, 0)$ y obtengo las distancias a todos los nodos, en específico a los nodos fantasma.

La distancia mínima a cada ciudad, será la distancia obtenida con Dijkstra a cada nodo fantasma de la respectiva ciudad.

Ejercicio 5). rayuela

rayuela de dimensiones $p \times q$ con números todos distintos escritos en los casilleros.

estando en (i, j) puedo pasar a las posiciones que estén en línea horizontal o vertical que (i, j) .

La casilla destino no puede estar a mayor distancia que k , y su número debe ser mayor al de la casilla (i, j) .

Al saltar de un casillero con número x a otro con número $x+r$, se gana r puntos.

El juego termina cuando no se pueden realizar más saltos válidos.

Encontrar el casillero inicial del cual puedo obtener la máxima cantidad de puntos.

solución.

armamos un grafo G cuyos vértices serán los pares (i, j) con $0 \leq i \leq p \wedge 0 \leq j \leq q$.

aristas: conectamos $v = (i_1, j_1)$ con $w = (i_2, j_2)$ con un eje de peso $r(w)$ si se cumple:

I_ v y w están en la misma línea horizontal o vertical y su distancia es menor que k .

o sea $w = (i_1 + k', j_1) \vee w = (i_1, j_1 + k')$

con $k' \leq k$.

II_ el valor del casillero w es mayor que el de v .

o sea $r(w) > r(v)$.

Para modelar que pueda empezar en cualquier posición y ver en dónde termina, agregamos un nodo fantasma inicio f_i y un nodo fantasma fin f_f .

conectamos a f_i con todos los nodos con costo 0, y a f_f con todos los nodos que tengan grado de salida 0, También con costo 0.

¡es un DAG!

armarlo cuesta $O(n+m)$.

$$\left. \begin{array}{l} n = p \cdot q \\ m = p \cdot q \cdot k \end{array} \right\} O(pq + pqk) = O(pqk) \text{ en el peor caso.}$$

para buscar el camino máximo de f_i a f_f como el algoritmo de camino máximo en DAG's visto en clase, lo cual me cuesta $O(n+m) = O(pqk)$

Dado el camino máximo hallado, como sabemos el casillero inicial, devolvemos el 2do nodo de este camino (pues como empezé de f_i , el 2do nodo corresponde al 1er casillero de donde voy a arrancar).

Ejercicio 6). canciones Floyd Warshall.

hay k acordes en total. La canción que quiero tiene n acordes. Me lleva S_{ij} segundos pasar del acorde i al acorde j . A veces conviene, para ir más rápido, tocar otros acordes entre el i y el j para llegar al j . ¿Cuál es el mínimo tiempo que me toma tocar la canción?

solución.

grafo de k vértices con una arista $i \rightarrow j$ de peso S_{ij} que sea $\forall$ par (i, j) el mínimo tiempo que necesito para pasar de i a j .

esto lo obtenemos en la matriz de distancias que genera el algoritmo de Floyd-Warshall.

Como Floyd Warshall es $O(n^3) = O(k^3)$.

El resultado será recorrer los n acordes de la canción y sumar las distancias correspondientes obtenidas en la matriz. $O(n)$

Complejidad: $O(k^3 + n)$.